ESP32 Firmware

How the NAZAR line controller runs the belt, reads the machine and fires the reject gate.

1. What the firmware does

NAZAR is a small inspection line. Washers ride a belt through a light tunnel; a camera on the Mac photographs each one and an AI model says good or bad. The ESP32 is the part that touches the machine. It has four jobs:

1. **Run the belt** through a relay, and stop it so each washer sits still under the camera.
2. **Read the machine**: motor voltage and current, vibration, light in the tunnel, motor temperature.
3. **Fire the reject gate** (a servo arm) when a bad washer reaches it.
4. **Talk to the Mac**: ten status lines a second, plus a message for every event, and it obeys short text commands such as `PASS`, `FAIL`, `HOLD`, `STOP`.

The exact wording of every message and command is fixed in `CONTRACT.md` in the hub repo (github.com/shubhammishra0212/nazar). The Mac console was written against the same file, so the two sides fit without being tested together first.

2. The ESP32

The board is an ESP32 DevKitC (WROOM module). Inside is a 240 MHz dual-core microcontroller with 520 KB of RAM and 4 MB of flash, running at 3.3 V. We chose it because it is cheap, the Arduino IDE supports it, it has hardware PWM on almost any pin (needed for the servo), a hardware I²C controller, and a USB-to-serial chip (CP2102) on the board, so one USB cable both programs it and carries the data. On the Mac it appears as `/dev/cu.usbserial-0001`.

The firmware is about 316 KB of flash (24 %) and 24 KB of RAM (7 %), so there is plenty of room.

3. How the code is organised

An Arduino sketch can have several files in its folder; they are all compiled together. Splitting the code means each person can read one topic at a time.

File	What it holds
`config.h`	Every pin, I²C address and default timing in one place. Change values here, nowhere else.
`nazar_line.ino`	`setup()` and `loop()`: wires the modules together. Short on purpose.
`line.h / line.cpp`	The state machine (RUN, INSPECT, HOLD, STOP), part counting, all line timers. No hardware calls at all.
`sensors.h / sensors.cpp`	INA219, MPU-6050, BH1750 (raw I²C) and DS18B20 (1-Wire).
`gate.h / gate.cpp`	The servo reject gate.
`belt.h / belt.cpp`	The belt relay.
`proto.h / proto.cpp`	Builds the JSON lines and reads the text commands.

Because `line.cpp` never touches a pin, it can be compiled on a laptop and tested with a fake clock (section 12). That is the most important design decision in the code.

4. The I²C bus

Three sensors share two wires: **SDA** (data, GPIO 21) and **SCL** (clock, GPIO 22). I²C is a bus: the ESP32 is the controller and each sensor has a fixed 7-bit address. To talk to one, the ESP32 sends its address; only that chip answers (an "ACK"). Our addresses:

Chip	Address
BH1750 light	0x23
INA219 power	0x40
MPU-6050 vibration	0x68

Both lines need pull-up resistors to 3.3 V; the sensor breakout boards already have them. Reading a sensor is always the same pattern: write the number of the register you want, then read some bytes back. We do this with the plain `Wire` library and three small helper functions instead of one library per sensor. Reasons: fewer downloads at the venue, less code to trust, and we know exactly which register is read.

Two safety settings: the bus runs at **100 kHz** (slower than the possible 400 kHz, but more tolerant of long jumper wires) and every transfer has a **20 ms timeout**, so a loose wire can never freeze the board.

Missing sensors. If a sensor does not answer at start-up, the firmware tries again every 3 seconds. If a working sensor fails 10 reads in a row, it is marked missing. While it is missing its values are simply left out of the status line and the Mac shows "No signal". When it comes back (wire pushed in again), a new `hello` message tells the Mac.

5. The sensors, one by one

INA219: belt motor voltage and current

The 5.5 V supply for the belt motor passes through a tiny 0.1 Ω resistor (the "shunt") on the INA219 board. Current through a resistor makes a small voltage across it (Ohm's law, V = I·R): 300 mA gives 30 mV. The INA219 measures that voltage very precisely, and also the voltage of the line itself. We set its configuration register to: 32 V range, ±320 mV shunt range, 12-bit, measuring continuously. Then:

- bus voltage register (0x02): shift right by 3, each step is 4 mV;
- shunt voltage register (0x01): signed, each step is 10 µV; current = shunt voltage ÷ 0.1 Ω.

Why it matters: a motor that is jammed or a belt that is too tight draws more current.

MPU-6050: vibration

The MPU-6050 is an accelerometer (plus gyro we do not use). At rest it measures gravity: 1 g. A shaking motor adds small fast changes on top. We read the three axes (x, y, z) 200 times a second and compute the size of the acceleration, $|a| = \sqrt{x^2 + y^2 + z^2}$.

We use the size and not one axis because our board is mounted tilted (about 77°); the size is the same whatever the angle. Then:

1. a slow running average of |a| tracks gravity (it follows changes over about half a second);
2. each sample's difference from that average is the vibration;
3. every 100 ms we report the **RMS** (root of the mean of the squares, the "average strength") as `vib`, and the largest single difference as `vpk`, both in g.

The chip identifies itself in register 0x75 (WHO_AM_I). A real one says 0x68; cheap clones say 0x70, 0x72 or 0x98 and work the same, so all four are accepted. Range is set to ±4 g so a hard knock on top of gravity does not clip.

BH1750: light in the tunnel

A digital light meter. We send "power on" and "continuous high-resolution mode" once; after that every 2-byte read is the latest measurement. Lux = raw value ÷ 1.2. The AI model was trained under a fixed light level, so if a tunnel LED fails, this number shows it before the AI starts making mistakes.

DS18B20: motor temperature

A digital thermometer on a different bus called **1-Wire** (one data wire on GPIO 4 with a 4.7 kΩ pull-up resistor). Here we use the OneWire and DallasTemperature libraries, because the 1-Wire timing is fiddly. One catch: at 11-bit resolution a measurement takes 375 ms, and the library normally waits for it. That would freeze the line for 375 ms. So we switch the waiting off (`setWaitForConversion(false)`), ask for a measurement, go on with everything else, and collect the result 400 ms later. A new reading arrives once a second; in between, the last value is repeated. A reading of exactly 85.0 °C is the chip's power-on value, not a real temperature, and is skipped.

FC-51: the part sensor

An infrared LED and a receiver side by side. A washer in front reflects the light back; a comparator on the board turns that into a clean digital signal: **LOW = part present**. The blue potentiometer sets the distance. Signals bounce for a few milliseconds when an edge of a washer passes, so the firmware only believes a new level after it has been steady for **20 ms** (debouncing). The moment the level becomes "present" is the rising edge that starts the cycle.

---

6. The reject gate: PWM and LEDC

A hobby servo (MG996R) is told its angle by a pulse repeated 50 times a second (every 20 ms). The pulse width sets the angle: about 0.5 ms = 0°, 2.5 ms = 180°. This is **PWM** (pulse width modulation). The ESP32 makes PWM in hardware with its **LEDC** peripheral, so the CPU does not spend any time on it. With esp32 core 3.x the call is `ledcAttach(pin, 50, 14)`: pin 13, 50 Hz, 14-bit resolution (16 384 steps per 20 ms, about 1.2 µs per step). The gate is closed at 0° and opens to 70° for `gate_open_ms` (600 ms by default); both angles are in `config.h`. Bring-up sketch 07 lets you find the right angles by typing them.

The servo needs its own 5-6 V supply with its ground joined to the ESP32 ground. Powering it from the ESP32 makes the board reset when the servo moves.

---

7. The belt relay: active-high (BC547 board)

The belt motor needs 5.5 V and up to an amp; the ESP32 pin gives 3.3 V and a few milliamps. A relay module bridges the gap: a small signal switches a mechanical contact in the motor line.

Our module is a simple "1-CH RELAY" board: IN goes through a resistor to a **BC547** (NPN) transistor that switches the 5 V coil. That makes it **active-high** — the team checked it by touching IN to 3.3 V (click) — and 3.3 V from the ESP32 is plenty to turn a BC547 on. So:

- **belt on** = pin is an OUTPUT at HIGH (3.3 V);
- **belt off** = pin is an OUTPUT at LOW (0 V).

`beltBegin()` is the first thing `setup()` does, so the pin is driven LOW (belt off) right after boot. While the board is still booting the pin is an input; the BC547 stays off with a floating base on a short wire, and a 10 kΩ resistor from IN to GND makes that certain.

`RELAY_ACTIVE_HIGH` in `config.h` switches back to the other common kind of module (optocoupler + H/L jumper on L): there LOW = on, and the pin is released to INPUT to switch it off, because 3.3 V HIGH is not always enough to release a module that pulls IN up to 5 V.

8. The line state machine

The line is always in one of four **modes**:

Mode	Belt	Meaning
RUN	on	normal running
INSPECT	off	a washer is stopped under the camera, waiting for the AI's verdict
HOLD	off	paused for an unsure part or by the operator, until `RESUME`
STOP	off	operator stop, until `START`; everything freezes exactly where it is

One washer's journey (default timings):

```
 IR sensor          camera                          gate
    |                 |                              |
----o-----------------o------------------------------o------>  belt
    |                 |                              |
 rising edge       belt stops                    gate opens
 -> "part"         -> "inspect"                  -> "gate"
    |<- stop_ms ->|   |<- dwell_ms .. wait_ms ->|  (only for FAIL)
      150 ms            1200 .. 5000 ms            |<- gate_ms ->|
                                      "go" + belt on     800 ms
```

```
          IR edge, stop_ms run
  +-----+ ---------------------> +---------+
  | RUN |                        | INSPECT |
  +-----+ <--------------------- +---------+
     ^    PASS / FAIL (after dwell_ms)  |
     |    or timeout (wait_ms)          | HOLD
     |                                  v
     |          RESUME             +---------+
     +---------------------------- |  HOLD   |   (part leaves as
     |   ("hold-released", no gate)+---------+    "hold-released")
     |
     |  STOP (from any mode)       +---------+
     +---------------------------- |  STOP   |   START returns to the
                                   +---------+   mode it came from
```

The verdict rules:

- `PASS`: the belt restarts once `dwell_ms` has passed since the stop.
- `FAIL`: the same, and a gate timer starts.
- no verdict by `wait_ms`: the part passes with `res:"timeout"`. The line never waits forever for the AI.
- `HOLD` while a washer is under the camera: it stays there until `RESUME`, then leaves as `hold-released` without the gate.

Running-time timers. The distance from camera to gate is fixed, so what matters is how long the belt has actually *run*, not how much clock time has passed. Every position timer (IR to camera, camera to gate) only counts down while the belt is running in normal mode. If the operator presses STOP when a bad washer is half-way to the gate, its timer freezes; after START it continues with the time that was left, and the gate still fires exactly when the washer gets there.

Queues. Several washers can be on the belt at once. There are 4 slots for parts between the IR sensor and the camera and 8 slots for rejected parts between the camera and the gate. Each has its own timer, so two bad washers close together are both caught.

Continuous mode (`SET stops 0`): the belt never stops for inspection. An `inspect` message is still sent when a part reaches the camera, and a `FAIL` gates the most recent part when it reaches the gate.

IR edges are ignored while the belt is off, because nothing can move then: a change is a hand, not a part.

9. Non-blocking timing

The board must do many things "at once": sample vibration 200 times a second, send status 10 times a second, watch the IR sensor, count timers, read commands. A `delay()` anywhere would make everything else wait. So the firmware never waits. `loop()` runs thousands of times a second; each job checks the clock (`millis()`, milliseconds since power-on) and either does its small piece of work or returns at once:

```
if (now - lastTel >= 100) { lastTel += 100; sendTelemetry(); }
```

`millis()` is a 32-bit number and wraps back to 0 after 49.7 days. Writing the test as `now - last >= interval` with unsigned numbers stays correct across the wrap; the logic test checks this.

10. The JSON-lines protocol

Every message from the ESP32 is one line of **JSON** (a text format for key/value data that every language can read). One message per line means the Mac can read line by line and ignore anything that is not valid JSON, such as the boot garbage the ESP32 prints after a reset. The `"t"` key says what kind of message it is:

```
{"t":"tel","ms":183402,"v":11.92,"ma":312.4,"vib":0.042,"vpk":0.110,"lux":812.5,"temp":34.6,"ir":0,"parts":17,"rej":3,"gate":0,"belt":1,"mode
{"t":"part","n":18}
{"t":"inspect","n":18}
{"t":"go","n":18,"res":"reject"}
{"t":"gate","n":18}
```

We build these strings by hand with `snprintf` (fill-in-the-blanks formatting) into a fixed buffer instead of using a JSON library: it is small, uses no dynamic memory, and the output is exactly what the contract shows.

Commands from the Mac are plain text, one per line, upper or lower case: `PING`, `PASS`, `FAIL`, `HOLD`, `RESUME`, `STOP`, `START`, `REJECT`, `BELT 1/0` (only in STOP), `RESET`, `SET <key> <number>`. Each is answered with `ack` or with `err` and the reason. The serial link runs at 115 200 baud (about 11 500 characters a second); the status lines use about 15 % of that.

---

11. Bring-up sketches

`bringup/` has eight tiny sketches, one per part, numbered in the order to wire and test them:

Sketch	What you should see
`01_i2c_scan`	lists 0x23, 0x40, 0x68 with PASS next to each
`02_ina219`	about 0 mA with the belt off, a few hundred mA with it on, about 5.5 V
`03_mpu6050`	WHO_AM_I, |a| about 1.00 g, vib jumps when you tap the table
`04_bh1750`	lux drops when you cover it with your hand
`05_ds18b20`	25-35 °C, rises when you hold the probe
`06_fc51_ir`	"PART n seen" for every washer you pass in front
`07_servo_gate`	type `o`, `c`, `s` or an angle; the arm follows
`08_relay_belt`	type `1` / `0`; the relay clicks and the belt runs / stops

Each prints plain sentences at 115 200 baud, with a hint when something is wrong (for example "no voltage on VIN-: is the 5.5 V rail on and S1 on?"). Testing parts one at a time finds a bad wire in minutes instead of hours.

---

12. How to flash and test

1. Install the **OneWire** and **DallasTemperature** libraries; board **ESP32 Dev Module**.
2. Compile and upload `nazar_line` (Arduino IDE, or the `arduino-cli` commands in the README). If the upload hangs at "Connecting...", hold the BOOT button.
3. Close the Serial Monitor and run `python3 tools/serial_check.py`. It sends PING, prints what arrives for 5 seconds, checks every line against the contract, lists which sensors are present and ends with `OK` or a list of problems. It can also send commands: `python3 tools/serial_check.py HOLD RESUME`.
4. Open the NAZAR Console.

Tests that run without any board:

- `make -C test` compiles the real `line.cpp` on the Mac with a fake clock and runs 11 scenarios (55 checks): PASS, FAIL with exact gate timing, STOP in the middle of a gate countdown then START, STOP during an inspection, timeout, HOLD then RESUME, two bad parts close together, continuous mode, debounce, the BELT rule, and the `millis()` wrap. As a check of the check, breaking the gate countdown on purpose makes 8 of the checks fail.
- `python3 tools/serial_check.py --selftest` tests the validator on good and bad sample lines and does a full PING round trip over a fake serial port.

---

13. What is verified and what is not

Verified (on the Mac, 2 Nov 2026):

- The main firmware and all eight bring-up sketches compile for the ESP32 (core 3.3.12) with all warnings on; none of the warnings are in our files.
- The state machine behaves as the contract says in every scenario above, to the millisecond.
- The checker tool parses and validates the contract's message formats.

Not yet verified on real hardware (no board was connected while writing):

- The sensor register settings and conversions (INA219 scale, MPU range, BH1750 lux) against real readings. They follow the datasheets and the previous NAZAR firmware.
- The servo angles 0° / 70° for our gate arm; tune with sketch 07.
- The relay switching from GPIO26 HIGH / LOW on the real BC547 board (checked by hand with IN to 3.3 V only).
- The real `stop_ms` and `gate_ms` for our belt speed and distances: measure, then send `SET stop_ms ...` / `SET gate_ms ...` from the console.
- Timing of the whole loop under load (MPU at 200 Hz with the DS18B20 read in the same second).

14. Choices the contract left open

- After a reboot the line starts in RUN with the belt on (`BOOT_STOPPED` in `config.h`).
- A manual `REJECT` reports `{"t":"gate","n":0}` ; every gate firing counts in `rej` .
- In continuous mode `inspect` is sent with the belt still running and no `go` follows.
- A verdict with no part waiting, a second verdict, or a verdict while the part is on HOLD is answered with `err` . `HOLD` , `RESUME` , `STOP` and `START` are always acknowledged.